Partition Crossover for Pseudo-Boolean Optimization

Renato Tinós
Department of Computing and Mathematics
University of São Paulo
Ribeirão Preto, SP, Brazil
rtinos@ffclrp.usp.br

Darrell Whitley
Department of Computer Science
Colorado State University
Fort Collins, CO, USA
whitley@cs.colostate.edu

Francisco Chicano
Department of Lenguajes y Ciencias de la Computación
University of Málaga
Málaga, Spain
chicano@lcc.uma.es

ABSTRACT

A partition crossover operator is introduced for use with NK landscapes, MAX-kSAT and for all k-bounded pseudo-Boolean functions. By definition, these problems use a bit representation. Under partition crossover, the evaluation of offspring can be directly obtained from partial evaluations of substrings found in the parents. Partition crossover explores the *variable interaction graph* of the pseudo-Boolean functions in order to partition the variables of the solution vector. Proofs are presented showing that if the differing variable assignments found in the two parents can be partitioned into q non-interacting sets, partition crossover can be used to find the best of 2^q possible offspring. Proofs are presented which show that parents that are locally optimal will always generate offspring that are locally optimal with respect to a (more restricted) hyperplane subspace. Empirical experiments show that parents that are locally optimal generate offspring that are locally optimal in the full search space more than 80 percent of the time. Experimental results also show the effectiveness of the proposed crossover when used in combination with a hybrid genetic algorithm.

Categories and Subject Descriptors

I.2.8 [**Artificial Intelligence**]: [Problem Solving, Control Methods, and Search]

General Terms

Algorithms, Theory

Keywords

Pseudo-Boolean optimization, NK Landscape, Recombination Operator, Partition Crossover, Evolutionary Combinatorial Optimization

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.
Copyright 20XX ACM X-XXXXX-XX-X/XX/XX ...$10.00.

1. INTRODUCTION

A **partition crossover** operator is introduced for recombinating solutions to k-bounded pseudo-Boolean functions. The set of k-bounded pseudo-Boolean functions includes MAX-kSAT problems, NK Landscapes and generalized spin glass problems. These problems are all NP Hard. All of these problems use a bit representation; the output can be any real valued number. Thus, a pseudo-Boolean function has the form: $f : \mathbb{B}^n \rightarrow \mathbb{R}$. If the function f is also k-bounded, f can be decomposed into m subfunctions, where each subfunction takes at most k bits as input. For example, in an NK Landscape, there are N variables and N subfunctions that take as input $K+1$ bits; thus, $m = N$ and $k = (K + 1)$. (Note that we will use small n when refering to problem size in general, and use capital N, where $N = n$, when specifically discussing NK Landscapes.) Heckendorn [11] has referred to the class of k bounded pseudo-Boolean functions as *embedded landscapes*. We could also generalize Kauffman's [16] concept of NK Landscapes and use the term "Mk Landscapes" to refer to all pseudo-Boolean functions which are a linear combination of m subfunctions, where each subfunction takes at most k bits as input.

The limitation to k bounded functions is not as restrictive as it might seem. Every search problem that uses a bit representation and which has an algebraic evaluation function can be converted into a k-bounded pseudo-Boolean function in polynomial time [2]. For example, one can construct examples where it is feasible to take an NK Landscape where $n = 5,000$ and $k = K + 1 = 6$ and to convert it into another NK Landscape where $n = 10,000$ and $k = 3$, where both problems have the same global optimum (or global optima). Of course, in such cases the actual search spaces that are induced will be different. The reduction of a general pseudo-Boolean optimization problem to a k-bounded problem is analogous to the reduction of a general SAT problem to a MAX-kSAT problem [5]. A reduction to CNF-SAT that is k-bounded is routine when solving SAT instances [7] [15].

Partition crossover decomposes the evaluation function as well as the set of variables during recombination. This decomposition can be done in such a way as to make it possible to evaluate offspring using partial evaluations obtained from the parents. For k bounded pseudo-Boolean functions, the set of bits with different assignments in two parent solutions can be partitioned into q non-overlapping subsets and the objective function can be written as a sum of q subfunctions; this is done in a manner such that each variable is included in only one of the new subfunctions. Because offspring are evaluated using the sum of the q partial eval-

uations of the parents, the best subsolution for each set of variables in the partition can be selected in a greedy fashion. Thus partition crossover deterministically returns the best of 2^q offspring. The parents are included in the set of possible offspring. If no offspring is better than the best parent, partition crossover returns the best parent.

Previous forms of partition crossover have been developed for the symmetric *Traveling Salesman Problem* (TSP) [31] [8] and the asymmetric TSP [29]; these operators have been shown to be highly effective and to yield state of the art results when combined with a powerful local search algorithm such as Lin-Kernighan-Helsgaun Local Search [13]. Each improvement to partition crossover (and generalized partition crossover) has increased the number of "recombining components" by producing a more fine grain decomposition of the parent solutions.

The Lin-Kernighan-Helsgaun implementation [13] already includes a recombination operator in the form of "Iterated Partial Transcription" [19]. Forms of partition crossover that cut more than two common edges for the TSP finds opportunities for recombination that are missed by Iterated Partial Transcription; during recombination Iterated Partial Transcription requires that "two subchains must have the same intial and final cities" (see [19]). Also, as far as we know, there are also no published cost complexity guarantees for Iterated Partial Transcription; in the worst case, this method appears to have $O(n^2)$ cost per recombination.

The TSP recombination operator proposed in [31], denoted *partition crossover* (PX), recombines partial solutions that are not shared in common between two parents. First, a union graph $G_u = G_x \cup G_y$ is created, where G_x and G_y are two graphs representing the two parent TSP solutions. Note that G_x and G_y represent Hamiltonian circuits over the set of vertices. The shared edges of G_u are removed and the connected components are identified using Breadth First Search (BFS). All the connected components separated from the rest of the graph by exactly two common edges and with more than one node are identified as *recombining components*. The remainder of the graph is also considered a recombining component. Partition crossover considers all recombinations between the recombining components to generate the offspring. In the TSP, the cost of evaluating one solution is $O(n)$. In the existing implementations of partition crossover operators, the partial evaluation of each one of the q recombining components are computed independently; this makes it possible to greedily select the best subsolution for each component. Therefore, existing partition crossover operators finds the best of 2^q possible offspring at cost $O(n)$. There can potentially exist forms of partition crossover with finer grained partitions and more recombining components, but finding such a partition could also have greater than $O(n)$ cost.

In the current paper the term *partition crossover* is used in a general sense to include both the original PX and the generalized PX operators (GPX and GAPX) for the TSP as well as a (generalized) partition crossover operator for pseudo-Boolean optimization problems. The original partition crossover (PX) for TSP used a single crossover point [31]; all subsequent versions of partition crossover uses multiple crossover points [8] [29].

We have already noted that partition crossover for the TSP has strong similarities to the "Iterated Partial Transcription" operator proposed by Möbius et al. [19]. Those authors also note that this idea could be applied in principle to spin glass problems by "flipping clusters" of a spin glass.

In this paper, the use of partition crossover for k-bounded pseudo-Boolean optimization problems is illustrated using NK landscapes. The NK landscape model [16] is widely used in different areas of research, such as theoretical biology, optimization, and physics. The number of local optima in an NK landscape is a product of the degree of epistasis K [20]. In the NK model, $k = (K + 1)$ different loci influence each one of the N subfunctions that compose the evaluation function. The evaluation function of the NK model is $f(x) \in \mathbb{R}$ where the binary vector x is epistatically bounded by a positive integer k.

Random NK Landscapes are considered to be difficult optimization problems for *genetic algorithms* (GAs) with standard recombination and mutation operators [12, 18, 1]. One problem is that crossover operators generally do not explore the linkage between the loci in the NK model. Therefore, the offspring is generally worse than the parents because recombination is disruptive.

One operator that considers linkage is network crossover [9]. Network crossover constructs a mask based on the epistasis matrix of the NK model. To build the crossover mask, a random bit of the solution vector is chosen as the starting point for randomized Breadth First Search. When the i-th bit of the solution vector is visited, the i-th bit of the mask is assigned 1, indicating that the i-th bit of the offspring is inherited from the first parent. When a desired number of bits are visited, the search is stopped. If the search is ended before the desired number of bits is reached, a new random starting point is chosen. Unlike network crossover, partition crossover uses the parents to partition the variable interaction graph in order to create a crossover mask.

The main contribution of the current paper is to prove the feasibility of partition crossover for k-bound pseudo-Boolean optimization problems, from both a theoretical and empirical point of view. We prove that partition crossover can be implemented as a deterministic and greedy crossover operator: it returns the best possible offspring given a particular decomposition of the parents. Partition crossover also returns an offspring which is provably locally optimal in a hyperplane subspace created by fixing the bits that are shared by the parents. The offspring is not guaranteed to be locally optimal in the original search space corresponding to the objective function f under the bit-flip neighborhood. However, empirical results indicate that offspring produced by partition crossover are often also locally optimal with respect to the original objective function f and the bit-flip neighborhood.

In our empirical experiments, offspring are locally optimal more than 80 percent of the time. This means that partition crossover is capable of "tunneling" from two parents that are local optima to a new offspring that is a new and distinct local optimum in a basin of attraction different from the basins containing the parents.

The decomposability of fitness functions by recombination is discussed in Section 2. The proposed crossover is presented and analyzed in Section 3. Experimental results in Section 4 show the efficiency of the proposed crossover when applied in combination with a hybrid GA. We discuss in Section 5 the "tunneling" capabilities of the new crossover operator and outline the conclusions and future work in Section 6.

2. DECOMPOSING THE EVALUATION FUNCTION

Let us first clarify the notation used for binary strings. We write here $\mathbb{B}^n$ as a synonym of $\mathbb{Z}_2^n$: $\mathbb{Z}_2^n = \mathbb{B}^n$. This set forms an Abelian group with the component-wise sum in $\mathbb{Z}_2$ (exclusive OR), denoted with $\oplus$. Given an element $z \in \mathbb{B}^n$, we will denote with $|z|$ the number of ones of z and with $\overline{z}$ the complement of the string (all bits inverted). We will denote with $\vee$ the bitwise OR operator between two binary strings and with $\wedge$ the bitwise AND. We will assume that the $\wedge$ operator has precedence over $\vee$. However, we will use parentheses to clarify the precedence. In order to simplify the notation, we will also use binary strings to represent sets of variables. In this case a binary string represents the set of variables having 1 in the string. For example, if the set of variables is $V = \{v_1, v_2, v_3, v_4\}$, the binary string 1010 can also be interpreted as the set $\{v_1, v_3\}$.

Assume that the objective function, f, is a k-bounded pseudo-Boolean function that is made up of m subfunctions. Let $\mathbb{B}^n$ represent the domain of function f: bit strings of length n. The objective function can be decomposed as follows:

$$f(x) = \sum_{i=1}^{m} f^i(x|_{mask_i}), \tag{1}$$

where f^i is a subfunction of f and $mask_i \in \mathbb{B}^n$ is used to denote the subset of variables needed to evaluate f^i. The notation $x|_{mask_i}$ is used to represent the restriction of x to the variables in $mask_i$; this way, with $f^i(x|_{mask_i})$ we are expressing the fact that function f^i only depends on the variables where $mask_i$ is 1. If the function is k-bounded, then each subfunction f^i accepts at most k bits as inputs, and $|mask_i| \leq k$.

Any discrete function $f : \mathbb{B}^n \to \mathbb{R}$ can be represented in the following polynomial form:

$$f(x) = \sum_{i \in \mathbb{B}^n} w_i \psi_i(x), \tag{2}$$

where w_i is a coefficient and $\psi_i(x) = (-1)^{i^T x}$ generates a sign. A discrete Fourier transform, also known as the Walsh transform, can be applied to f as follows:

$$w = \mathbf{W}f, \qquad f = \mathbf{W}^{-1}w,$$

where in the general case $\mathbf{W}$ is the $2^n \times 2^n$ matrix

$$\mathbf{W}_{ij} = \psi_i(j) = (-1)^{i^T j}.$$

If the function f is a linear combination of k-bounded subfunctions, then each subfunction f^i depends on k variables. Thus, for subfunction f^i the corresponding Walsh matrix is a $2^k \times 2^k$ matrix that transforms each subfunction in a Walsh polynomial. Thus each subfunction f^i contributes at most 2^k nonzero Walsh coefficients to the Walsh polynomial of $f(x)$ [10, 11].

First, the Walsh transform is applied to each subfunction over k bits:

$$w^i = \mathbf{W}f^i,$$

where w^i denotes the Walsh transform of f^i. Then, for any k-bounded pseudo-Boolean function we can generate the Walsh polynomial for the function f by linearly combining the coefficients obtained from each subfunction f^i as follows:

$$w = \sum_{i=1}^{m} w^i.$$

The vectors w^i are summed in the normal way, by adding coefficients with the same indices. If there are m subfunctions, the number of nonzero Walsh coefficients for f is at most $m\ 2^k$. If m is of size $O(n)$, the number of nonzero Walsh coefficients is also $O(n)$.

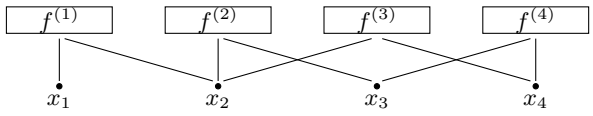
For example, for any NK Landscape, there are at most 2^{K+1} Walsh coefficients in each subfunction, $k = K + 1$ of which are linear, and one of which is a constant offset (w_0). Thus there are at most $N * (2^{K+1} - K - 2)$ nonlinear interactions. Thus, for fixed K there are $O(n)$ nonlinear interactions. For a MAX-3SAT problems when $m = 4n$, there are at most $2^3 - 4$ nonlinear Walsh coefficients per clause. Thus there are at most $4m = 16n$ nonlinear interactions. For any fixed k, as long as $m = O(n)$ there are at most $O(n)$ nonlinear interactions.

We should note there is nothing special about the use of Walsh coefficients. But the Walsh coefficients represent a precise way of identifying the nonlinear interaction between variables. We can also express k-bounded pseudo-Boolean functions as multilinear functions; a multilinear function would also uniquely identify nonlinear interactions between variables [2].

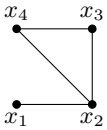
Definition 1 (Variable Interaction). *Given a pseudo-Boolean function $f : \mathbb{B}^n \to \mathbb{R}$ whose Walsh decomposition is given by Eq. (2), we say that variables v_j and v_k of f* interact*, if there exists a nonzero Walsh coefficient w_i in the Walsh decomposition of f such that $v_j, v_k \in i$. The* variable interaction *relationship is symmetric.*

The definition of *variable interaction* is completely general and does not depend on the form of the objective function. If the objective function can be written as a sum of k-bounded epistasis functions, like in Eq. (1), then two variables that interact will necessarily appear together as arguments of one of the subfunctions. For this reason, it is sometimes more convenient to say that two variables interact when they appear together in one of the subfunctions (see [4] for example). Often these two ways of defining interactions are exactly the same. However, the two concepts of interaction can differ, in the sense that two variables that interact according to the "subfunctions criterion" might not interact according to the "Walsh criterion." For example, assume a function includes two subfunctions f^l and f^r and that $f^l(x) = -f^r(x)$ and thus $f^l(x) + f^r(x) = 0$. Variables common to f^l and f^r interact under the "subfunction criterion" but not under the more precise "Walsh criterion" (unless the variables also appear together in other subfunctions). Unless otherwise stated, in this paper we will use the concept of variable interaction given in Definition 1 using the Walsh decomposition.

Definition 2 (Variable Interaction Graph). *Given a pseudo-Boolean function $f : \mathbb{B}^n \to \mathbb{R}$, we define the* Variable Interaction Graph *(VIG) of f as the graph $G = (V, E)$, where V is the set of variables of f and the edge set E contains all the pairs of variables (v_j, v_k) that interact.*



(a) Function



(b) Variable Interaction Graph

Figure 1: A function with $k = 2$ bounded epistasis, $n = 4$ variables and $m = 4$ subfunctions (top) and its corresponding Variable Interaction Graph (bottom). We assume here that none of the corresponding Walsh coefficients are zero.

In Figure 1 we show a function with k-bounded epistasis and its corresponding VIG.

PROPOSITION 3. *The maximum number of edges in the Variable Interaction Graph of a k-bounded pseudo-Boolean function f that can be written as a sum of m subfunctions is at most $mk(k-1)/2$.*

PROOF. Each subfunction depends on k variables and it can contribute at most $k(k-1)/2$ edges to the VIG. Since there are m subfunctions, the result follows. □

Let us now introduce some notation related to graphs. Given a graph $G(V, E)$ and a set of vertices $v \subseteq V$ of the graph, we define the *adjacency* of v, $adj_G(v)$, as the set of vertices that are adjacent in the graph to a vertex in v, that is:

$$adj_G(v) = \{w \in V | (\{w\} \times v) \cap E \neq \emptyset\}.$$

We define the *extended adjacency* of v as

$$adj^*_G(v) = adj_G(v) \cup v.$$

We will denote with $G[v]$ the *vertex-induced subgraph* of G containing only the vertices in v and the edges with both ends in v.

PROPOSITION 4. *Let $f : \mathbb{B}^n \to \mathbb{R}$ be a pseudo-Boolean function with VIG G, V the set of variables of f and v a subset of V. It is possible to additively decompose f as a sum of two functions g^1 and g^2, the first one depending on the variables in v and the second one not depending on the variables in v:*

$$f(x) = g^1(x|_{mask_1}) + g^2(x|_{mask_2}), \tag{3}$$

where $v \subseteq mask_1$ and $v \cap mask_2 = \emptyset$. The minimal set of variables a function g^1 must depend on is $adj^*_G(v)$.

PROOF. Let us define g^1 as the sum of the terms in the Walsh decomposition of f containing Walsh coefficients that include variables in v in their index:

$$g^1(x) = \sum_{\substack{i \in \mathbb{B}^n \\ i \cap v \neq \emptyset}} w_i \psi_i(x), \tag{4}$$

and g^2 is simply the sum of the other terms: $g^2(x) = f(x) - g^1(x)$. By the definition of g^2 the sum (3) holds. By the definition of g^1 we have that all terms depending on variables in v are included in g^1, so we have $v \subseteq mask_1$ and $v \cap mask_2 = \emptyset$.

A variable v' is in $adj_G(v)$ if there exists a nonzero Walsh coefficient (and term) with index including v' and a variable in v. However, we included all these Walsh terms in g^1, so it depends on all the variables in $adj^*_G(v)$. We can also notice that only the variables in $adj^*_G(v)$ are arguments of g^1.

Finally, we must prove that we need g^1 to depend on all the variables in $adj^*_G(v)$ if we want to additively decompose f in two functions with $v \subseteq mask_1$ and $v \cap mask_2 = \emptyset$. All the Walsh terms of f must be in one of the two functions, g^1 or g^2. In order to reduce the variables that g^1 depends on, we have to move one Walsh term from g^1 to g^2. But, by the definition of g^1, the moved term depends on a variable in v, and g^2 would depend on that variable. This is a contradiction, so $adj^*_G(v)$ is the minimal set g^1 must depend on. □

Given a subset of variables v of f, we would like to find a partition π of v such that f can be additively decomposed into $|\pi|$ subfunctions g^i for $i = 1$ to $|\pi|$, where each g^i depends on the variables in the subset π_i but does not depend on the variables in $v - \pi_i$. This kind of decomposition allows the variables in each subset π_i to be optimized independently from the variables in another subset. In particular, it will be used in partition crossover to select the parent providing the value for these variables. The next proposition provides the most fine grain partition on v.

PROPOSITION 5. *Let $f : \mathbb{B}^n \to \mathbb{R}$ be a pseudo-Boolean function with VIG G, V the set of variables of f and v a subset of V. The most fine grain partition π of v that additively decomposes f is given by:*

$$f(x) = h(x) + \sum_{i=1}^{|\pi|} g^i(x|_{mask_i}), \tag{5}$$

where $\pi_i \subseteq mask_i$, $(v - \pi_i) \cap mask_i = \emptyset$ and h does not depend on any variable in v, is the partition induced by the connected components in the subgraph $G[v]$.

PROOF. In the following we will focus on the variables of v. The subfunctions will depend on some other variables of $V - v$ but we are not interested in these variables. Proposition 4 implies that each g^i should contain at least one connected component of $G[v]$. We can prove this by contradiction. If g^i depends on a proper subset of a connected component t, by Proposition 4, it must also depend on $adj^*_G(t)$, but since t is in a connected component of $G[v]$, $adj^*_G(t)$ contains a variable of v which is not in t and will also be an argument of another subfunction g^j. Thus, if we want the arguments of the subfunctions g to be disjoint sets of variables of v we have to include complete connected components of $G[v]$ in each π_i.

We are interested in the most fine grain partition of v which decomposes the evaluation function f. The argument of the previous paragraph implies that the most fine grain partition must be the one induced by the connected components of $G[v]$. The subfunctions g^i can be defined from the Walsh decomposition of f as follows:

$$g^i(x) = \sum_{\substack{j \in \mathbb{B}^n \\ j \cap \pi_i \neq \emptyset}} w_j \psi_j(x). \tag{6}$$

The subfunction h, which does not depend on any variable of v is:

$$h(x) = f(x) - \sum_{i=1}^{|\pi|} g^i(x). \quad (7)$$

□

Although we explicitly included a function h in the additive decomposition of f that does not depend on any variable in v, sometimes we are not interested in that function and we just assume that it is summed to any of the other g^i subfunctions. Thus, in the following we will often say that f can be written as a sum of $|\pi|$ subfunctions and not $|\pi|+1$.

3. PARTITION CROSSOVER

The main recombination operators for binary strings in the literature are *respectful* and *transmit alleles* [23] [22]: that is, all "alleles" (e.g. bits) are inherited from the parents. Partition crossover is not an exception: all the bits shared in common by the two parent solutions will be inherited by the offspring and the differing bits will be *transmitted* from one parent or the other. The key characteristic of partition crossover is that the decision as to which parent to copy for each individual bit will be based on an additive decomposition of f. This will allow the operator to select the best offspring from among an exponential number of possible offspring.

Proposition 5 defines the most fine grain way to additively decompose f when we are interested in separating the variables in a subset v. In partition crossover the set of variables v in which we are interested is the set of differing variables in the two parent solutions. Thus, instead of the VIG of the function f, it will be useful in the following to work with a reduced graph: the *recombination graph*.

Definition 6 (Recombination Graph). *Let $f : \mathbb{B}^n \to \mathbb{R}$ be a pseudo-Boolean function with Variable Interaction Graph G and $x, y \in \mathbb{B}^n$ two solutions. We define the recombination graph of function f for solutions x and y as $G[x \oplus y]$, that is the subgraph of G composed of the variables in which x and y differ.*

In Algorithm 1, partition crossover (PX) is presented for pseudo-Boolean optimization problems. Prior to applying partition crossover it is necessary to construct the VIG, which is common to all the recombinations for the same objective function. Thus, it is provided as input in Algorithm 1.

The first step of the operator is to transfer all common bits to the offspring. This is done in line 1 of Algorithm 1. Observe that this operation can be written as the bitwise AND of the two parents. The AND of the differing bits will be zero.

Next, using the two parents x and y, determine the recombination graph and use Breadth First Search to find the connected components of the recombination graph (line 2). These connected components form a partition of the set of variables in $x \oplus y$ (the differing bits in the parent solutions).

Having determined the partition of variables in $x \oplus y$, test the two assignments from the two parents. Note that since this only includes non-shared assignments to variables, the solutions from the two parents will be complements. Proposition 5 ensures that we can write f as a linear sum of subfunctions g^i where each one depends on the variables in one component π_i and not the others (line 3). Select the assignment in each connected component that results in the best evaluation for the corresponding subfunction g^i (lines 4 to 10).

Algorithm 1 Partition Crossover (assuming maximization)

Input: f (pseudo-Boolean function)
Input: G (Variable Interaction Graph of f)
Input: x, y (parent solutions)
Output: z (offspring)
1: $z \leftarrow x \wedge y$ // Transfer the common bits
2: $\pi \leftarrow$ computePartition $(G[x \oplus y])$
3: Let $f = \sum_{i=1}^{|\pi|} g^i$ according to Proposition 5.
4: **for** $i = 1$ to $|\pi|$ **do**
5: **if** $g^i(x) > g^i(y)$ **then**
6: $z \leftarrow z \vee (x \wedge \pi_i)$
7: **else**
8: $z \leftarrow z \vee (y \wedge \pi_i)$
9: **end if**
10: **end for**

Theorem 7. *For any k-bounded pseudo-Boolean function f, if the recombination graph of f for solutions x and y contains q connected components, then partition crossover returns the best of 2^q solutions, including the parent solutions.*

Proof. Since partition crossover is considering the q connected component independently and there are two possible sources of bits (the two parents), it is exploring a subspace of 2^q solutions, which include the parent solutions. Since f can be written as a linear sum of subfunctions, each one depending on the variables of each connected component, the best solution in this subspace is obtained by optimizing each subfunction independently, that is, selecting the best parent for each connected component. This is what partition crossover does, so it must return a solution whose evaluation is better than or equal to the best of the 2^q solutions. □

From the previous theorem it is clear that a larger number of connected components implies the exploration of a larger set of solutions. In the experimental section we will study how many connected components can be found in some instances of NK landscapes. Despite the exponential number of solutions explored during crossover, the complexity of the operator is linear in n, as the next theorem states.

Theorem 8. *If the number of nonzero Walsh coefficients of f is $O(n)$ and the epistasis degree k is a constant, partition crossover runs in $O(n)$ time.*

Proof. The bitwise AND of line 1 requires $O(n)$ because n is the length of the binary strings. The BFS algorithm of line 2 requires $O(|E|)$, but $|E|$ is bounded by the number of nonzero Walsh coefficients, which is $O(n)$. Evaluating all the subfunctions g^i in lines 4 to 10 and assigning the corresponding values to z also requires $O(n)$ because the g functions are composed of disjoint sets of Walsh terms and there are only $O(n)$ nonzero of such terms. □

We will use an NK landscape to illustrate how the VIG and the recombination graph are used to find a partition that can be exploited by recombination. Assume we are given the following NK landscape, where $n = 8$ and $K = 2$ (and $k = K+1 = 3$). In the following table, $mask_i$ indicates

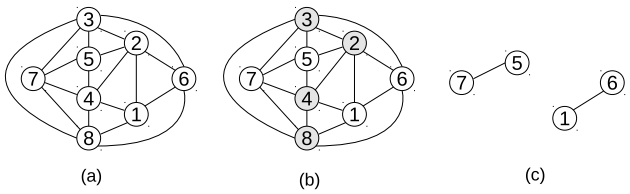


Figure 2: The VIG is shown on the lefthand side (a). When recombining the solutions 00000000 and 10001110, the vertices and edges associated with shared variables are deleted (b) to yield the "recombination graph" shown on the righthand side (c). If the recombination graph can be partitioned into q connected components, then recombination is guaranteed to return the best of 2^q possible offspring.

which bits are utilized by subfunction f^i. Bits are numbered from left to right, from 1 to 8.

$mask_1 = 8, 4, 1 = 10010001$	$mask_2 = 5, 3, 2$
$mask_3 = 6, 3, 2 = 01100100$	$mask_4 = 4, 2, 1$
$mask_5 = 7, 5, 4 = 00011010$	$mask_6 = 8, 6, 1$
$mask_7 = 7, 5, 3 = 00101010$	$mask_8 = 8, 7, 3$

The Walsh transformation of this NK landscape generates the following order-2 and order-3 Walsh coefficients (let us assume they are all nonzero):

f^1 yields $w_{1,4}, w_{1,8}, w_{4,8}, w_{1,4,8}$
f^2 yields $w_{2,5}, w_{2,3}, w_{5,3}, w_{2,3,5}$
f^3 yields $w_{3,6}, w_{3,2}, w_{6,2}, w_{2,3,6}$
f^4 yields $w_{4,2}, w_{4,1}, w_{2,1}, w_{1,2,4}$
f^5 yields $w_{5,7}, w_{5,4}, w_{7,4}, w_{4,5,7}$
f^6 yields $w_{6,8}, w_{6,1}, w_{8,1}, w_{1,6,8}$
f^7 yields $w_{7,3}, w_{7,5}, w_{3,5}, w_{3,5,7}$
f^8 yields $w_{8,7}, w_{8,3}, w_{7,3}, w_{3,7,8}$

This results in the VIG shown on the lefthand side of Figure 2.

Assume we wish to recombine two solutions 00000000 and 10001110; again, the variables are numbered from 1 to 8 from left to right. These two solutions reside in the hyperplane subspace *000***0 because these zero bits are shared in common by the parents. Because "respectful" recombination inherits all of the variable values that the two solutions share in common, recombination must generate a solution in this same subspace.

From the VIG, we will generate the recombination graph. Partition crossover first deletes those variables that share a common value assignment from the VIG. The edges connected to the variables with common values are also deleted. This yields the recombination graph. We next ask if deleting these variables and edges "partitions" the recombination graph. In our example, the variables with shared values are 2, 3, 4 and 8. The variables that have different assignments are 1, 5, 6 and 7. The recombination graph is partitioned into two independent subgraphs, as illustrated in Figure 2(c).

Since, the recombination graph partitions into two independent subgraphs, by Proposition 5 this also implies that the objective function can be partitioned into two *new subfunctions* that can be linearly combined to evaluate any and all offspring produced from the parents by partition crossover.

The following results pertain to questions about the "optimality" of offspring generated by partition crossover. We will define a function f' in relationship to a specific recombination of x and y as:

$$f'(z) = f((x \wedge y) \vee (z \wedge (x \oplus y))), \tag{8}$$

that is, if there are s common bits between x and y, f' accepts as input the $n - s$ non-common bits. Its value will be the same as f where non-common bits are taken from z and the common bits are taken from x and y. The function f' is the restriction of f to a hyperplane subspace.

3.1 Locally Optimal Parents

In practice, we will only apply partition crossover to parents that are also local optima with respect to the Hamming distance 1 neighborhood. The reason for this restriction is due to the fact that we need parents to have some degree of similarity in order for partitions to exist. For example, if we tried to recombine two random solutions, x_1 and x_2 we might expect x_1 and x_2 to share approximately one half of the bit assignments to variables. If two solutions share more than one half of the same bit assignments, then the recombination random is more likely to partition the VIG, because more vertices are removed from the VIG when the recombination graph is constructed. Thus, partition crossover is limited to local optima because local optima are more likely to share more than one half of the same bit assignments. We could also explicitly decide which local optima to recombine by picking those that share more bits in common. This idea will be revisited in section 4 of this paper.

THEOREM 9. *Given a pseudo-Boolean function f, two solutions x and y that are local optima of f using the 1-flip neighborhood, and the function f' defined in (8), the offspring generated by partition crossover will be local optima in the function f' using the 1-flip neighborhood. However, the offspring need not always be a local optimum in f, but an improved offspring is guaranteed to be in a new basin of attraction different from that of its parents.*

PROOF. Because the parents are local optima, by definition there is no improving move under a bit flip in either parent that can yield an improving move. Because the offspring results from changes in the non-fixed variables in separable components (and separable subfunctions), and they share exactly the same shared variable assignments of the parents, the offspring cannot be improved by a bit flip that changes any of the variables that appear in any of the components. If such an improving move did exist, then the same improving move would exist in one of the parents, and thus one of the parents is not really locally optimal. This leads to a contradiction. Thus the offspring must be local optima in f'.

However, assume that an offspring is generated that has one variable v_a that changes in component π_i and another variable v_b changes in a different component π_j, leading to an assignment combination that did not previously exist in the parents. Such a new combination of variables assignment can lead to a potential improving move in the evaluation

function f. Again assume the function is expressed in terms of the Walsh coefficients: let v_z be a variable with the same bit assignment in both parents and assume there exists a Walsh coefficient $w_{a,c,z}$ and $w_{b,d,z}$. Assume variables v_a and v_c are in π_i, and v_b and v_d are in π_j. Note that v_z must not be in the recombination graph for these variables to be in different components (otherwise there would be one single connected component containing v_a, v_b, v_c and v_d). Assume recombination generates an overall assignment to v_a, v_c, v_b and v_d that is not found in either parent, but which is better than either parent; the potential contribution of $w_{a,c,z}$ and $w_{b,d,z}$ is now different. The combination of assignments to variables v_a, v_b, v_c, v_d, v_z is at least 3 bit flips away from either parent when v_z is flipped, and thus a single bit flip (of v_z) can now potentially correspond to a new improving move.

If the new offspring is better than both parents, and the parents are local optima, then the new offspring must be in a new basin of attraction under the 1-flip neighborhood. □

This theorem makes it clear that the only bit flips that can lead to additional improving moves after partition crossover requires that local search flips a variable that had the same assignment in both parents.

In general, assume the recombination graph is partitioned into q connected components. We can, of course, always explicitly compute the Walsh coefficients, then partition the coefficients to construct q partial evaluation functions. But we can also just treat the shared bits in the subfunctions as being fixed and partition the original subfunctions. Both approaches compute the same result, and both allow a greedy method to find the best subsolution in each partition to be determined in $O(n)$ time.

THEOREM 10. *Given two locally optimal parents, at least two connected components of the recombination graph must contain more than 1 vertex for recombination to find an improving move.*

PROOF. Assume there is only one component with size greater than 1; denote this by π_L. Next assume there is one or more components of size 1. Assume the assignment from component π_L is inherited first from parent x (without loss of generality). At this point the offspring has inherited all the shared variables from parent x and all of the variables from π_L from parent x. Since all of the other components are of size 1, these moves correspond to moves in the Hamming distance 1 neighborhood. Therefore we know that the offspring must be in the same basin of attraction as parent x because all of the other components are of size 1, and the associated subfunctions g are all linear because only one variable in $x \oplus y$ contributes to each partial evaluation. Therefore, if parent x is a local optimum, all of these components with only one variable that differs from x must be disimproving or neutral moves. □

THEOREM 11. *If there are at least two "larger components" containing more than 1 vertex, and each locally optimal parent contributes improving assignments to bits in one of these "larger components," then additional components of size 1 can also contribute to improvements of the offspring.*

PROOF. Assume the offspring inherits bits m, n, o from parent x in a larger component, and bits p, q, r from parent y in another larger component; both must be improving assignments. Since both parents have contributed to the offspring, and the offspring is the best of the 2^q solutions (which includes the parents), the offspring must be better than the parents. Because the parents are local optima, the offspring must reside in a new basin of attraction. The components of size 1 still behave like Hamming distance one moves, but since the offspring is in a different basin of attraction than the parents, these Hamming distance 1 moves can correspond to new improving moves that have not already been exploited by local search. □

4. EXPERIMENTS

The proposed partition crossover (PX) operator can be applied as a recombination operator in a wide range of different search algorithms that might be used to optimize an NK landscape instance. Here, we compare partition crossover to two other crossover operators in a hybrid GA. Our goal in the current paper is not to compare to the state of the art, but rather to explore the behavior of partition crossover empirically.

4.1 Hybrid Genetic Algorithm

The hybrid GA is described in Algorithm 2. Bit-flip local search with first improvement is applied to random solutions in order to generate the initial population. Elitism is used to preserve the current best solution, while tournament selection with a pool of 5 solutions is used to select parents for crossover and mutation.

We also tested an alternative approach for selection in combination with partition crossover. The first parent is chosen by tournament selection. The second parent is the closest individual (in terms of Hamming distance) to the first parent among the solutions in the same tournament pool used to select the first parent. Only individuals that are different are selected. Therefore, the first parent is chosen based only on fitness, while the second parent is chosen based on the distance to the first parent. This increases the number of bit assignments the two parents share in common. We denote this selection mechanism as *fit/dist selection*. Our hypothesis is that we can decompose parents into more independent components by selecting parents that share more bits in common. In the limit this fails: if parents share all the bits in common, there is no partition and no components.

One of the limitations of all *respectful* and *transmitting* recombination operators is that crossover can only reach parts of the search space composed of the alleles present in the current population. Some method is needed to generate new diversity. Every 10 generations of the GA, 90% of the solutions of the current population are replaced by random solutions optimized by bit-flip local search with first improvement. The same local search is also applied to the remaining 10% of the solutions, including the best solution found in the previous generation. Flip mutation is employed with mutation rate equal to $\frac{3}{n}$. The other parameters of the GA are: population size (n_{pop}) equal to 50 individuals, number of runs equal to 50, crossover rate (p_{cross}) equal to 0.6, and the execution time for each run is $\frac{n(k-1)}{25}$ seconds.

The hybrid GAs with PX are here compared to 2 other algorithms: the same hybrid GA with *2-point crossover* [25] [6] and the same hybrid GA with *Uniform crossover* [26]. All algorithms are exactly the same, except that they use different crossover operators (and, in case of GA with PX and

Algorithm 2 Hybrid GA

```
1: for i=1 to n_pop do
2:    P(i)=randomIndividual();
3:    P(i)=LSFI(P(i)); // Local Search with First Improvement
4: end for
5: while termination condition is not satisfied do
6:    Q(1)=bestSolution(P);
7:    for i=2 to n_pop do
8:       (p1,p2)=selection(P);
9:       if random() < p_cross then
10:         Q(i)=crossover(p1,p2);
11:      else
12:         Q(i)=mutation(p1);
13:      end if
14:   end for
15:   if every 10 gen. then
16:      Q(1)=bestSolution(Q);
17:      for i=1 to 0.1 * n_pop do
18:         Q(i)=LSFI(Q(i));
19:      end for
20:      for i=0.1*n_pop+1 to n_pop do
21:         Q(i)=randomIndividual();
22:         Q(i)=LSFI(Q(i));
23:      end for
24:   end if
25:   P=Q;
26: end while
27:
28: Comments (selection):
29: i) Both parents for GA with PX, GA with 2-point crossover, GA with Uniform crossover and first parent for GA with
    PX and fit/dist selection: tournament selection, i.e., the selection is based only on fitness
30: ii) Second parent for the GA with PX and fit/dist selection: choose the solution with the smallest Hamming distance
    (larger than 0) to parent 1 among the solutions in the pool used to choose parent 1, i.e., selection based on distance to
    parent 1.
31: Comments (crossover and mutation):
32: iii) The fitness of the offspring Q(i) is computed after the application of mutation
33: iv) For GA with 2-point crossover and GA with Uniform crossover, the fitness of the offspring Q(i) is computed after
    the application of crossover. For the GAs with PX, the fitness of the offspring Q(i) is returned by the proposed method
    together with the solution.
```

fit/dist selection, a different selection method for the second parent). This means that differences in the results between algorithms with 2-point crossover, Uniform crossover, and PX are only due to the effectiveness of recombination. Differences in the results of the hybrid GA with PX and the hybrid GA with PX and fit/dist selection are due to the selection operator. While the former uses standard tournament selection, the latter uses the fit/dist selection.

4.2 Results

The algorithms were applied to NK landscapes with $n = \{100, 300, 500\}$ and with $K = \{1, 2, 3\}$. The *adjacent* and *random* epistasis models were considered. In the adjacent NK model, subfunction f^i uses bit x_i and K additional bits adjacent to bit x_i. This creates a regular simple cascade of interaction that makes it possible to use exact methods to obtain the globally optimal solution for the adjacent NK model in polynomial time [32].

For the adjacent NK landscape problems, Table 1 presents the percentage of the runs that found the global optimum. The results were obtained over 50 runs. The table also presents the average difference in evaluation between the global optimum and the best solution when the global optimum is not found. The problems were posed as maximization problems. Given a solution, s' and the global optimum s^*, the percentage of difference is calculated as:

$$\%Difference(s') = 100(f(s^*) - f(s'))/f(s^*).$$

The use of partition crossover (PX) clearly improves the genetic algorithm's ability to reach the global optimum for all instances. The GA using PX found a global optimum on every run, except in the instances with $N = 300, K = 3$; $N = 500, K = 2$ and $N = 500, K = 3$. The GA using partition crossover with fit/dist selection found the global optimum 70 percent of the time in the worst case. When $N = 500, K = 3$, partition crossover used together with fit/dist selection found far more globally optimal solutions than using partition crossover and only fitness-based selection. 2-point crossover and Uniform crossover were not even close to being competitive on the adjacent NK landscape problems. Still, 2-point crossover yielded better results than Uniform crossover in terms of locating globally optimal solutions, in particular, for the smallest problems, where $N = 100$. This occurs because Uniform crossover is

Table 1: Percentage over 50 runs where the global optimum was found (Found) in the experiments of the hybrid GA with the adjacent model. The average percentage difference (% Difference) with respect to the global optimum evaluation is also given.

		2-point crossover		Uniform crossover		PX		PX fit/dist selection	
N	K	Found	% Difference	Found	% Difference	Found	% Difference	Found	% Difference
100	1	100	0.000 ±0.000	88	0.011 ±0.051	100	0.000 ±0.000	100	0.0000 ±0.0000
100	2	90	0.007 ±0.029	24	0.294 ±0.347	100	0.000 ±0.000	100	0.0000 ±0.0000
100	3	62	0.086 ±0.192	4	0.657 ±0.432	100	0.000 ±0.000	100	0.0000 ±0.0000
300	1	18	0.090 ±0.087	0	0.859 ±0.262	100	0.000 ±0.000	100	0.0000 ±0.0000
300	2	0	0.611 ±0.212	0	2.157 ±0.431	100	0.000 ±0.000	100	0.0000 ±0.0000
300	3	0	1.503 ±0.402	0	3.464 ±0.506	80	0.009 ±0.023	98	0.0001 ±0.0007
500	1	0	0.364 ±0.153	0	1.371 ±0.307	100	0.000 ±0.000	100	0.0000 ±0.0000
500	2	0	1.398 ±0.327	0	3.261 ±0.377	98	0.001 ±0.004	98	0.0011 ±0.0078
500	3	0	2.791 ±0.467	0	4.851 ±0.518	40	0.029 ±0.042	70	0.0079 ±0.0183

Table 2: Mean evaluation (over 50 runs) for the hybrid GA with the random model. The evaluation functions is being *maximized*. A Wilcoxon signed rank test was used to compare the GA with PX and fit/dist selection against all of the other GAs. The symbol "=" indicates null hypothesis (the results of the algorithms are equal) cannot be rejected at the 0.05 significance level. The symbol "s" indicates the algorithm finds significantly poorer solutions compared to the hybrid GA with PX and fit/dist selection.

N	K	2-point crossover	Uniform crossover	PX	PX and fit/dist selection
100	1	0.71577 ±0.01619 (s)	0.71587 ±0.01613 (s)	0.71593 ±0.01613 (=)	0.71593 ±0.01613
100	2	0.75105 ±0.01337 (s)	0.75093 ±0.01389 (s)	0.75180 ±0.01353 (=)	0.75201 ±0.01345
100	3	0.76886 ±0.01038 (=)	0.76937 ±0.01031 (=)	0.76927 ±0.01013 (=)	0.76899 ±0.01046
300	1	0.70965 ±0.01054 (s)	0.71005 ±0.01119 (s)	0.71519 ±0.01051 (=)	0.71519 ±0.01051
300	2	0.73926 ±0.00856 (s)	0.73923 ±0.00929 (s)	0.74649 ±0.00923 (=)	0.74657 ±0.00974
300	3	0.75402 ±0.00689 (s)	0.75350 ±0.00555 (s)	0.75444 ±0.00647 (s)	0.75653 ±0.00699
500	1	0.70470 ±0.00765 (s)	0.70493 ±0.00722 (s)	0.71422 ±0.00739 (=)	0.71422 ±0.00739
500	2	0.73061 ±0.00644 (s)	0.73054 ±0.00623 (s)	0.73865 ±0.00671 (s)	0.74022 ±0.00684
500	3	0.74400 ±0.00517 (s)	0.74423 ±0.00440 (s)	0.74528 ±0.00545 (=)	0.74548 ±0.00509

very disruptive for the NK landscapes, even for the adjacent model.

Table 2 presents the average evaluations obtained in the experiments for the random NK landscapes. A Wilcoxon signed rank test was used to compare the GA with PX and fit/dist selection against all of the other GAs. The reason why we use the signed rank test is because each of the 50 runs of the algorithms is performed on a different NK landscape instance and a different initial population. However, the i-th run of each algorithm solves the same instance and starts with the same population. That is, the samples are paired, and Wilcoxon signed rank test is appropriate. The GA with PX and fit/dist selection was significantly better than using 2-point and Uniform crossover except for the instance $N = 100, K = 3$ where Uniform crossover was best, but the differences were not significant. This probably is due to the ability of Uniform crossover to widely explore the search space, setting the stage for local search to explore different parts of the search space. For example, there could be an isolated region of the search space containing the global optimum or highly competitive local optima that are not easily reached by partition crossover. It is notable that the case where Uniform crossover was best was on a smaller problem. Again, this suggests randomness in the NK landscape instance and not a failure to scale up to larger problems (as indicated by the size of N) and greater nonlinearity (as indicated by the size of K).

Another interesting question is how many connected components can be found in the recombination graph, that is, what is the cardinality of the partitions. Recall that if there are q components, then partition crossover will return the best of 2^q distinct offspring. Table 3 shows the mean, standard deviation and the maximum number of *components* for each application of partition crossover. This table shows two interesting trends. First, as expected, the number of components is higher for adjacent NK landscapes than for random NK landscapes. Second, the number of components is higher for lower values of K. Both observations suggest that partition crossover is sensitive to problem structure, which should have a positive impact on search.

Table 4 shows the mean number of improvements that result from crossover during the first 10 generations. We limited the data to the first 10 generations because if an algorithm is able to quickly locate a good solution, that algorithm will produce fewer improvements in the later stages of search. The data in Table 4 shows that Uniform crossover very rarely results in an improved solution. This suggests that the main benefit of Uniform crossover is not in discovering improved solutions, but rather in moving the search to a new location that can be explored by other operators. Uniform crossover is strongly biased toward exploration instead of exploitation. 2-point crossover is sometimes effective at finding improving moves on adjacent NK landscapes, but appears to be no better than Uniform crossover on random NK landscapes. Partition crossover is dramatically better at finding improved offspring than Uniform and 2-

Table 3: Mean (and standard deviation) of the number of components for each application of crossover and the maximum number of components found by the algorithms in the experiments with the hybrid GAs.

N	K	Model	PX		PX and fit/dist selection	
			Mean numb. of comp.	Max. comp.	Mean numb. of comp.	Max. comp.
100	1	Adjacent	0.46 ±0.09	14	3.34 ±0.16	16
100	2	Adjacent	0.58 ±0.05	13	3.45 ±0.08	15
100	3	Adjacent	0.48 ±0.02	11	3.12 ±0.04	11
300	1	Adjacent	1.50 ±0.19	33	5.37 ±0.35	35
300	2	Adjacent	1.84 ±0.10	31	6.08 ±0.19	34
300	3	Adjacent	1.49 ±0.05	24	5.24 ±0.10	26
500	1	Adjacent	2.72 ±0.27	48	7.66 ±0.47	55
500	2	Adjacent	3.29 ±0.17	49	8.81 ±0.24	51
500	3	Adjacent	2.75 ±0.12	39	7.52 ±0.16	41
100	1	Random	0.40 ±0.09	13	3.22 ±0.16	15
100	2	Random	0.18 ±0.02	8	2.48 ±0.05	12
100	3	Random	0.13 ±0.00	7	2.14 ±0.02	11
300	1	Random	1.33 ±0.19	29	5.05 ±0.35	34
300	2	Random	0.33 ±0.05	12	2.77 ±0.08	16
300	3	Random	0.16 ±0.01	9	2.41 ±0.04	13
500	1	Random	2.44 ±0.27	43	6.98 ±0.47	47
500	2	Random	0.50 ±0.06	14	2.97 ±0.11	15
500	3	Random	0.18 ±0.02	10	2.46 ±0.05	13

point crossover. The number of improvements is higher for adjacent NK landscapes. For random NK landscapes, the number of improvements resulting from crossover decreases as K increases. This does not happen on the adjacent NK landscapes.

5. TUNNELING BETWEEN OPTIMA

Previous work on the Traveling Salesman Problem (TSP) showed that the offspring generated by *generalized partition crossover* were locally optima in the majority of cases. This means that recombination can move directly from 2 known local optima to many other local optima. We have observed on 2000 city TSP problems that one recombination can reach thousands (e.g., $q = 12$ generating 2^{12} offspring) and even millions (e.g., $q = 25$) of other local optima.

In the case of the Traveling Salesman Problem, the reason that the offspring is usually locally optimal is as follows: there can be no improving move *within* a connected tour segment after the graph unioning the two parents has been partitioned to yield the "recombination graph." Assume the local search operator is 2-opt, which reverses a single segment of the Hamiltonian circuit. There can only be improving moves using 2-opt as long the begin and end of the segment being reversed by 2-opt are not contained within a connected tour segment after the graph is partitioned.

For k-bounded pseudo-Boolean optimization problems, our theoretical results show that the offspring returned by partition crossover must be locally optimal in the hyperplane subspace in which the two parents have differing bits. The local search operator in this case is the bit-flip neighborhood, but this observation extends to any Hamming ball neighborhood of radius r in the following more restricted sense: if there is an improving move involving 1, 2, or any number of bit flips up to r bit flips that falls into the *same* connected component, that move would have already been exploited when local search was applied to the parents.

Assuming partition crossover finds an improved offspring, if the Hamming distance 1 neighborhood is used, any additional improvement in the offspring must be the result of flipping at least one variable assignment that was shared in common by the parents.

In practice, how often are the offspring produced by partition crossover locally optima? We will conduct an experiment using Algorithm 3 to explore this question empirically. In this experiment, only crossover is applied to recombine the best current solution to optimal solutions generated by local search first improvement. The population size is equal to 20 individuals, the number of runs is 50, and the number of generations is 20 (termination condition).

Algorithm 3 Recombination of Local Optima

```
1: while termination condition is not satisfied do
2:    for i=1 to maxpop do
3:       P(i)=randomIndividual();
4:       P(i)=LSFI(P(i));
5:    end for
6:    if f(bestSolution(P)) > f(x*) then
7:       x*=bestSolution(P);
8:    end if
9:    for i=1 to maxpop do
10:      Q(i)=crossover(x*,P(i));
11:   end for
12:   if f(bestSolution(Q)) > f(x*) then
13:      x*=LSFI(bestSolution(Q));
14:   end if
15: end while
```

Recall that parents are always locally optimal in our hybrid genetic algorithm. Also, recall that a new and different offspring is generated by partition crossover only if the offspring is better than the parents. Table 5 shows the percentage of offspring (different from the parents) that were locally optimal. Results are shown for 2-point crossover, Uniform crossover and partition crossover. As one would expect, Uniform crossover almost never produced offspring that were locally optimal.

Table 4: Mean number of improvements, during the first 10 generations, in the best fitness caused by crossover in the experiments with the hybrid GA.

N	K	Model	2-point crossover	Uniform crossover	PX	PX and fit/dist selection
100	1	Adjacent	6.04 ±4.23	1.26 ±1.58	26.84 ±13.48	22.78 ±11.25
100	2	Adjacent	9.90 ±4.61	0.84 ±1.35	46.94 ±15.22	38.26 ±12.33
100	3	Adjacent	7.46 ±4.60	0.56 ±0.79	55.68 ±15.61	42.76 ±12.69
300	1	Adjacent	16.04 ±6.14	0.60 ±0.88	60.50 ±11.23	57.64 ±11.11
300	2	Adjacent	16.50 ±7.07	0.46 ±0.68	87.02 ±15.64	91.12 ±13.06
300	3	Adjacent	14.60 ±5.77	0.80 ±1.12	103.40 ±15.37	104.12 ±24.72
500	1	Adjacent	16.94 ±7.53	0.66 ±0.96	75.64 ±11.64	77.74 ±10.49
500	2	Adjacent	18.64 ±7.67	0.44 ±0.50	104.92 ±15.99	110.74 ±14.23
500	3	Adjacent	15.08 ±6.16	0.36 ±0.48	126.80 ±18.35	130.86 ±17.82
100	1	Random	1.20 ±1.31	1.76 ±1.92	25.62 ±11.84	21.86 ±10.58
100	2	Random	0.52 ±1.05	0.58 ±1.07	14.68 ±10.74	9.64 ±6.01
100	3	Random	0.52 ±0.79	0.50 ±0.71	4.12 ±5.46	4.18 ±3.89
300	1	Random	0.86 ±1.09	0.78 ±1.07	59.74 ±12.75	52.00 ±12.76
300	2	Random	0.62 ±0.83	0.74 ±1.08	24.12 ±16.48	22.98 ±10.58
300	3	Random	0.40 ±0.57	0.64 ±0.98	4.00 ±4.66	4.88 ±4.62
500	1	Random	0.62 ±0.90	0.70 ±1.16	81.38 ±16.83	73.96 ±12.77
500	2	Random	0.62 ±0.75	0.60 ±0.76	31.32 ±16.81	34.14 ±15.55
500	3	Random	0.58 ±0.81	0.62 ±0.92	5.08 ±5.25	5.58 ±5.31

The 2-point crossover operator is better than expected at generating offspring that are locally optimal when the problem is an adjacent NK landscape problem. The probability that 2-point crossover generates an offspring that is locally optimal decreases as K increases. For these adjacent problems, 2-point crossover has some of the properties of partition crossover, since variable interactions (like those exploited by partition crossover) are such that interacting variables are close to each other on the representation. If both crossover points used by 2-point crossover fall into regions of shared bit assignments, 2-point crossover can implicitly exploit some of the benefits and behaviors of partition crossover. Note that 2-point crossover can only isolate 2 components and *the components must be contiguous.* Also, 2-point crossover is not greedy. And so while 2-point crossover may generate offspring that are locally optimal more often than expected on adjacent NK landscape problems (where the interacting bit are contiguous), it does not generate nearly as many improving moves as partition crossover, as can be seen in Table 4.

The ability of 2-point crossover to generate offspring that are locally optimal disappears on the random NK landscape problems. When the interactions between variables is not localized, 2-point crossover cannot perform the complex decomposition that is performed by partition crossover.

Partition crossover generates offspring that are locally optimal on both adjacent NK landscapes and random NK landscapes. The probability of generating locally optimal offspring does decrease somewhat as N and K increase, but the change is not sudden or dramatic. For the adjacent NK landscapes the percentage of locally optimal offspring ranged from 93% to 100%. For the random NK landscapes the percentage of locally optimal offspring ranged from 83% to 100%. Partition crossover is surprisingly effective at "tunneling" from parents that are local optima to improved offspring that are also locally optimal.

6. CONCLUSIONS AND FUTURE WORK

A partition crossover operator is introduced that can be used for any k-bounded pseudo-Boolean optimization problem. Partition crossover breaks the set of n variables into q subsets. These q subsets of variables are made of variables that have different assignments in the two parents. The subsets of variables are partitioned such that each subset only interacts with other variables in the same subset. This makes it possible to inherit the best possible assignment of variables for each subset. Thus, given a partition of the variables into q subsets, partition crossover can return the best of 2^q offspring.

Our theoretical results show that the offspring returned by partition crossover must be locally optimal in the hyperplane subspace in which the two parents have differing bits. In practice the offspring generated by partition crossover are also usually locally optimal in the full search space as well. In the experiments presented in this paper offspring were locally optimal more than 83 percent of the time on random NK landscape problems with $N = 500$ and $K = 3$.

Empirical tests indicate that partition crossover yields significantly better results than 2-point crossover or Uniform crossover on NK landscape problems when K is relatively small.

The results presented in this paper are not intended to establish a new best known algorithm for NK landscapes or for other k-bounded pseudo-Boolean optimization problems. Pelikan et al. [21] compare evolutionary algorithms and estimation of distribution algorithms using the adjacent NK landscape model. The Linkage Tree Genetic Algorithm has also displayed good performance on adjacent NK landscape problems [27] [28]. Whitley and Chen [30] have introduced a highly efficient local search method that can identify improving moves in k-bounded pseudo-Boolean optimization problems in constant time without enumerating the neighborhood. Chen et al. [3] studies how these methods scales up as K increases for NK landscapes.

Chicano, Whitley and Sutton [4] show that under special

Table 5: Mean percentage of offspring different from the parents that are local optima for the experiments of recombination of local optima. Note that in the case of the PX operator, every offspring is also an improving move.

N	K	Model	2-point crossover	Uniform crossover	PX
100	1	Adjacent	74.22 ±3.99	0.39 ±0.36	100.00 ±0.00
100	2	Adjacent	47.69 ±4.56	0.01 ±0.04	95.45 ±8.36
100	3	Adjacent	30.05 ±3.87	0.00 ±0.00	97.44 ±4.47
300	1	Adjacent	77.10 ±2.81	0.00 ±0.00	97.64 ±6.66
300	2	Adjacent	50.23 ±3.29	0.00 ±0.00	95.33 ±5.86
300	3	Adjacent	30.74 ±2.87	0.00 ±0.00	94.42 ±4.36
500	1	Adjacent	78.01 ±2.36	0.00 ±0.00	97.93 ±5.02
500	2	Adjacent	50.10 ±3.24	0.00 ±0.00	93.84 ±5.91
500	3	Adjacent	31.04 ±2.53	0.00 ±0.00	93.87 ±4.03
100	1	Random	0.77 ±0.95	0.57 ±0.58	100.00 ±0.00
100	2	Random	0.05 ±0.11	0.02 ±0.07	93.16 ±10.71
100	3	Random	0.00 ±0.00	0.00 ±0.00	93.67 ±16.11
300	1	Random	0.02 ±0.06	0.00 ±0.00	97.28 ±6.38
300	2	Random	0.01 ±0.04	0.00 ±0.00	85.65 ±7.60
300	3	Random	0.01 ±0.05	0.00 ±0.00	86.46 ±17.19
500	1	Random	0.03 ±0.08	0.00 ±0.00	98.33 ±4.92
500	2	Random	0.00 ±0.00	0.00 ±0.00	84.83 ±5.64
500	3	Random	0.00 ±0.00	0.00 ±0.00	83.65 ±16.81

conditions (e.g, the number of times that variables appear in subfunctions must be relatively uniform), a local search algorithm can be constructed to identify improving moves in radius r Hamming ball neighborhoods. Again, this is done without enumerating the neighborhood and the calculation has constant time complexity with respect to n. (The computation is sensitive to the radius r and the degree of epistasis k.) Optimal solutions have been generated for 10,000 variable adjacent NK landscape problems by looking 7 moves ahead (e.g. $r = 7$).

The results presented in this paper can potentially be combined with the results of Chicano, Whitley and Sutton [4] by combining a Hybrid Genetic Algorithm that uses some local search with smaller Hamming ball radius (e.g. $r = 3$) and partition crossover. Under these conditions, can partition crossover combined with iterated local search find the global optimum (or highly competitive solutions) using a small Hamming ball radius neighborhood?

The results in the current paper and the work of Chicano, Whitley and Sutton [4] do not deal with plateaus. The existence of plateaus is an extremely important part of the search space landscape for MAX-kSAT problems. Specialized algorithms for MAX-kSAT, such as WalkSAT [24] and other modern algorithms based on local search [14] [17] are highly effective; what makes these algorithms so effective is the strategies they employ to select moves when the search has reached a flat plateau. Future work will need to evaluate the degree to which partition crossover is effected by plateaus in the local search neighborhood.

7. ACKNOWLEDGMENTS

Renato Tinós was a visiting researcher in the Department of Computer Science at CSU. The visit of the researcher was made possible by the support provided by FAPESP (under grant 2012/22200-9), USP, and CSU. This research was sponsored by the Air Force Office of Scientific Research, Air Force Materiel Command, USAF (under grant FA9550-11-1-0088). The U.S. Government is authorized to reproduce and distribute reprints for Governmental purposes notwithstanding any copyright notation thereon. Francisco Chicano was a Fulbright Visiting Scholar at CSU supported by the Fulbright program, the Spanish Ministry of Education ("José Castillejo" mobility program), the Universidad de Málaga, Andalucía Tech, and CSU.

8. REFERENCES

[1] H. Aguirre and K. Tanaka. A study on the behavior of genetic algorithms on NK-landscapes: Effects of selection, drift, mutation, and recombination. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 86(9):2270–2279, 2003.

[2] E. Boros and P. Hammer. Pseudo-boolean optimization. *Discrete applied mathematics*, 123(1):155–225, 2002.

[3] W. Chen, D. Whitley, D. Hains, and A. Howe. Second order partial derivatives for nk-landscapes. In *Proceeding of the fifteenth annual conference on Genetic and evolutionary computation conference*, GECCO '13, pages 503–510, New York, NY, USA, 2013. ACM.

[4] F. Chicano, D. Whitley, and A. Sutton. Efficient identification of improving moves in a ball for pseudo-boolean problems. In *Proceedings of the 16th annual Conference on Genetic and Evolutionary Computation*. ACM, 2014.

[5] T. Cormen, C. Leiserson, and R. Rivest. *Introduction to Algorithms*. McGraw Hill, New York, 1990.

[6] D. Goldberg. *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley, Reading, MA, 1989.

[7] C. Gomes, H. Kautz, A. Sabharwal, and B. Selman. Satisfiability solvers. In *Handbook of Knowledge Representation*, pages 89–134. Elsevier, 2008.

[8] D. Hains, D. Whitley, and A. Howe. Revisiting the big valley search space structure in the TSP. *Journal of the Operational Research Society*, 62(2):305–312, 2011.

[9] M. W. Hauschild and M. Pelikan. Network crossover performance on NK landscapes and deceptive problems. In *Proc. of the 12th Annual Conference on Genetic and Evolutionary Computation*, pages 713–720, 2010.

[10] R. Heckendorn and D. Whitley. A walsh analysis of NK-landscapes. In *Proceedings of the Seventh International Conference on Genetic Algorithms*, 1997.

[11] R. B. Heckendorn. Embedded landscapes. *Evolutionary Computation*, 10(4):345–369, 2002.

[12] R. B. Heckendorn, S. Rana, and D. Whitley. Test function generators as embedded landscapes. *Foundations of Genetic Algorithms*, 5:183–198, 1999.

[13] K. Helsgaun. An effective implementation of the lin–kernighan traveling salesman heuristic. *European Journal of Operational Research*, 126(1):106–130, 2000.

[14] H. Hoos. An adaptive noise mechanism for WalkSAT. In *Proceedings of the National Conference on Artificial Intelligence*, pages 655–660, 2002.

[15] H. H. Hoos and T. StÃijtzle. SATLIB: An online resource for research on SAT. In I. Gent and H. M. T. Walsh, editors, *SAT 2000*, pages 283–292, 2000.

[16] S. A. Kauffman. *The origins of order: Self-organization and selection in evolution*. Oxford university press, 1993.

[17] L. Kroc, A. Sabharwal, C. Gomes, and B. Selman. Integrating systematic and local search paradigms: A new strategy for maxsat. In *Proceedings of the 21st international jont conference on Artifical intelligence*, pages 544–551, 2009.

[18] K. E. Mathias, L. J. Eshelman, and D. Schaffer. Niches in NK-landscapes. *Foundations of Genetic Algorithms*, 6:27–46, 2001.

[19] A. Möbius, B. Freisleben, P. Merz, and M. Schreiber. Combinatorial optimization by iterative partial transcription. *Physical Review E*, 59(4):4667–4674, 1999.

[20] G. Ochoa, M. Tomassini, S. Vérel, and C. Darabos. A study of NK landscapes' basins and local optima networks. In *Proc. of the 10th Annual Conference on Genetic and Evolutionary Computation*, pages 555–562, 2008.

[21] M. Pelikan, K. Sastry, D. Goldberg, M. Butz, and M. Hauschild. Performance of evolutionary algorithms on nk landscapes with nearest neighbor interations and tunable overlap. In *Proc. of the Genetic and Evolutionary Computation Conference (GECCO)*, 2009.

[22] N. Radcliffe and P. Surry. Fitness variance of formae and performance predictions. In D. Whitley and M. Vose, editors, *Foundations of Genetic Algorithms 3*, pages 51–72. Morgan Kaufmann, 1995.

[23] N. J. Radcliffe. The algebra of genetic algorithms. *Annals of Maths and Artificial Intelligence*, 10:339–384, 1994.

[24] B. Selman, H. Kautz, and B. Cohen. Noise strategies for improving local search. In *Proceedings of the National Conference on Artificial Intelligence*, pages 337–337, 1994.

[25] W. Spears and K. D. Jong. An Analysis of Multi-Point Crossover. In G. Rawlins, editor, *Foundations of Genetic Algorithms*, pages 301–315. Morgan Kaufmann, 1991.

[26] G. Syswerda. Uniform Crossover in Genetic Algorithms. In J. D. Schaffer, editor, *Proc Third International Conf on Genetic Algorithms*. Morgan Kaufmann, 1989.

[27] D. Thierens. The Linkage Tree Genetic Algorithm. In *Parallel Problem Solving from Nature*, pages 264–273. Springer Lecture Notes in Computer Science, 2010.

[28] D. Thierens and P. Bosman. Evolvability Analysis of the Linkage Tree Genetic Algorithm. In *Parallel Problem Solving from Nature*, pages 286–295. Springer Lecture Notes in Computer Science, 2012.

[29] R. Tinós, D. Whitley, and G. Ochoa. Generalized asymmetric partition crossover for the traveling salesman problem. In *Proc. of the 16th Annual Conference on Genetic and Evolutionary Computation*, 2014.

[30] D. Whitley and W. Chen. Constant time steepest descent local search with lookahead for NK-landscapes and MAX-kSAT. In *GECCO*, pages 1357–1364, 2012.

[31] D. Whitley, D. Hains, and A. Howe. Tunneling between optima: partition crossover for the TSP. In *Proc. of the 11th Annual Conference on Genetic and Evolutionary Computation*, pages 915–922, 2009.

[32] A. H. Wright, R. K. Thompson, and J. Zhang. The computational complexity of N-K fitness functions. *IEEE Transactions on Evolutionary Computation*, 4(4):373–379, 2000.